A D PATEL INSTITUTE OF TECHNOLOGY
THE CHARUTAR VIDYA MANDAL
DEPT. OF INFORMATION TECHNOLOGY

MINI PROJECT REPORT

ON

PERSONAL EXPENSE TRACKER

Submitted in partial fulfilment of the requirements of the course

PROGRAMMING WITH PYTHON

In

Information Technology

Semester – V

By:

Parmar Dwija

(12502080603005)

Under the Guidance of

Nayan Mali

Department of Information Technology

A D Patel Institute of Technology

Academic Year 2026–2027

A D PATEL INSTITUTE OF TECHNOLOGY
THE CHARUTAR VIDYA MANDAL
DEPT. OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project report entitled “Personal Expense Tracker” has been prepared and submitted by Parmar Dwija, Enrollment No. 12502080603005, a student of B.Tech. in Information Technology, Semester 5, at A. D. Patel Institute of Technology, CVM University, during the academic year 2026–2027.

The project demonstrates the use of Python programming concepts including Lists, Dictionaries, File Handling, and Tkinter. It is designed to help users record, organize, view, and manage their personal expenses through a simple graphical user interface.

This report is submitted as part of the academic requirements for the B.Tech. in Information Technology programme. The project work has been carried out under the guidance and supervision of the undersigned.

Date: _____

Project Guide	Head of Department
Nayan Mali	Dr. Priyank Bhatt Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to A. D. Patel Institute of Technology, CVM University, for providing me with the opportunity to develop the project entitled “Personal Expense Tracker” as part of my B.Tech. in Information Technology curriculum.

I am thankful to my project guide, Nayan Mali, for valuable guidance, encouragement, and support throughout the development of this project. The suggestions and feedback helped me understand the project requirements and improve the implementation.

I would also like to thank the faculty members of the Information Technology Department for their assistance and for providing the academic knowledge required to complete this project.

I am grateful to my friends and family for their encouragement and support during the development and documentation of this project.

Finally, I acknowledge the developers of Python and Tkinter and the contributors to the Python documentation, which helped me understand the tools and concepts used in this project.

Parmar Dwija
(12502080603005)

ABSTRACT

Personal Expense Tracker is a Python-based desktop application designed to help users record and manage their everyday expenses in an organized manner. Keeping track of daily spending manually can be time-consuming and may make it difficult to understand where money is being spent.

The application uses Python for program logic and Tkinter to create a graphical user interface. Lists and dictionaries are used to organize expense records and their related details, while file handling allows expense information to be saved to and loaded from a file so that records can be retained between sessions.

The system is intended to provide simple functions such as adding an expense, viewing saved expenses, categorizing spending, calculating the total amount spent, and deleting an entry when required. The interface is designed to be easy to understand and operate.

The project demonstrates the practical application of core Python concepts—Lists, Dictionaries, File Handling, and Tkinter—in a small real-world problem. It can be extended in the future with monthly summaries, charts, budget limits, search filters, and export options.

TABLE OF CONTENTS

Certificate 2

Acknowledgement 3

Abstract 4

Table of Contents 5

1. Introduction and Problem Statement 6

2. Objectives and Scope 8

3. Technologies Used and System Requirements 10

4. System Design and Implementation 12

5. Features, Testing and Results 14

6. Conclusion, Future Scope and References 16

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

Managing personal finances is an important part of everyday life. People make different kinds of payments for food, travel, shopping, education, bills, and other needs. Without a proper record, it can be difficult to remember each expense and calculate total spending.

Personal Expense Tracker is a desktop application developed using Python. It provides a simple graphical interface through which users can enter expense details, organize records by category, view previously saved entries, and calculate the total amount spent.

The application uses Lists to maintain a collection of expense records and Dictionaries to represent each expense through fields such as date, category, description, and amount. File Handling is used to save and retrieve records, while Tkinter provides windows, labels, text fields, buttons, and a table-like display.

1.2 Problem Statement

When expenses are recorded only in memory or on separate pieces of paper, users may lose track of spending. Manual calculations can also lead to errors and make it difficult to review past transactions.

The major problems include:

- Difficulty in keeping all expense records in one place.
- Forgetting small daily purchases and payments.
- Time-consuming manual calculation of total expenses.
- Limited ability to organize expenses by category.
- Risk of losing handwritten records or unsaved information.

1. INTRODUCTION AND PROBLEM STATEMENT (CONTINUED)

Therefore, a simple application is useful for storing expense information in a structured format and making it easy to view and calculate spending. Personal Expense Tracker addresses this need with a lightweight desktop interface and local file storage.

1.3 Proposed Solution

The proposed system lets a user enter an expense, validate the amount, and save the record. The application displays the stored entries and updates the total whenever a record is added or removed. Records are written to a local file and loaded when the application starts.

1.4 Significance of the Project

This project connects classroom Python concepts to a practical daily-life problem. It shows how basic data structures can represent information, how file operations preserve data, and how a GUI can make a program more accessible to non-technical users.

2. OBJECTIVES AND SCOPE

2.1 Objectives

The main objective of Personal Expense Tracker is to develop a simple, user-friendly desktop application that helps users record expenses and understand their spending.

1. Expense Entry: Allow users to enter expense details such as date, category, description, and amount.
2. Expense Organization: Store expense records in a structured list and represent each record using a dictionary.
3. Persistent Storage: Save expense data to a file and load it when the application starts.
4. Total Calculation: Calculate the total amount of all recorded expenses.
5. Expense Review: Display saved records so users can review their spending.
6. Record Management: Provide a way to remove a selected expense when needed.
7. Simple Interface: Use Tkinter widgets to provide a clear and convenient interface.

2.2 Scope of the Project

The project is intended for individual users who want a basic local tool for recording everyday expenses. It focuses on core expense-management features and does not require an online account or a remote database.

2. OBJECTIVES AND SCOPE (CONTINUED)

The scope of the project includes the following functions:

- Adding an expense with a date, category, description, and amount.
- Displaying expense entries in a list or table-like view.
- Calculating and displaying the total of recorded expenses.
- Saving records to a local data file using file handling.
- Loading saved records when the application is reopened.
- Deleting a selected record and updating the displayed total.

2.3 Limitations

The basic version is designed for local use and may not include user authentication, cloud synchronization, automatic bank transaction import, or advanced analytics. The safety of saved data depends on the local file and the user's device.

2.4 Future Scope

The application can be enhanced by adding monthly budgets, category-wise summaries, graphical charts, search and filter options, CSV export, password protection, and automatic backup. These features can improve the usefulness of the system while retaining its simple interface.

3. TECHNOLOGIES USED AND SYSTEM REQUIREMENTS

3.1 Technologies Used

1. Python

Python is the primary programming language used to develop the application. It handles user input, expense calculations, validation, data organization, and file operations.

2. Tkinter

Tkinter is Python's standard interface to the Tcl/Tk GUI toolkit. It provides widgets such as windows, labels, entry fields, buttons, frames, and message boxes that allow users to interact with the program.

3. Lists

A list is an ordered, mutable collection in Python. In this project, a list can hold all expense records, allowing the application to add, display, iterate through, and remove entries.

4. Dictionaries

A dictionary stores data as key-value pairs. Each expense can be represented by keys such as date, category, description, and amount. This makes individual fields easier to access and update.

5. File Handling

File handling is used to store records beyond the current program session. The application can open a file, read existing expense data, write updated records, and close the file. A consistent format such as JSON can be used for structured storage.

3. TECHNOLOGIES USED AND SYSTEM REQUIREMENTS (CONTINUED)

3.2 Software Requirements

- Operating System: Microsoft Windows 10/11 or a compatible desktop OS
- Programming Language: Python 3.x
- GUI Library: Tkinter (usually included with standard Python installations)
- Code Editor: Visual Studio Code, IDLE, or another Python editor
- Data Storage: Local text or JSON file

3.3 Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB recommended
- Storage: At least 100 MB free space for source code and data
- Display: Standard laptop or desktop display
- Input Devices: Keyboard and mouse or touchpad

3.4 Development Environment

The application can be developed in a Python environment using a code editor such as Visual Studio Code. Tkinter is used for the graphical interface, and local file storage is used to preserve expense records. No internet connection is required for basic operation.

4. SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Design

Personal Expense Tracker follows a simple desktop application structure. The user interacts with the Tkinter interface, Python functions process the entered information, and file handling stores or retrieves expense records.

8. Presentation Layer: Tkinter widgets display input fields, action buttons, expense records, and total spending.
9. Application Logic Layer: Python functions validate input, add or remove records, and calculate totals.
10. Data Layer: A local file stores the expense records for later retrieval.

4.2 System Workflow

11. The user launches the Personal Expense Tracker application.
12. The application reads saved records from the local file, if it exists.
13. The interface displays saved expenses and the current total.
14. The user enters the date, category, description, and amount.
15. The program validates the input and creates a dictionary for the new expense.
16. The dictionary is added to the expense list and the file is updated.
17. The interface refreshes the records and recalculates the total.

4.3 Data Structure Design

A list is used as the main collection of expense records. Each record is represented by a dictionary containing related fields. For example, one record may contain date, category, description, and amount. This approach keeps the data organized and allows each field to be accessed by its key.

4. SYSTEM DESIGN AND IMPLEMENTATION (CONTINUED)

4.4 File Handling

When the application starts, it checks whether the data file exists. If it exists, the saved records are read and converted into Python data structures. When a record is added or deleted, the updated list is written back to the file. Error handling should be used for missing files, invalid data, or permission problems.

4.5 Tkinter Interface

The graphical interface can include a main window, title label, input fields, category selection, Add Expense and Delete Expense buttons, an expense table, and a total amount label. Tkinter event handlers connect button clicks to Python functions.

4.6 Implementation Logic

The add-expense function collects user input, checks that required fields are present, converts the amount to a numeric value, creates an expense dictionary, appends it to the list, saves the data, and refreshes the display. The delete function identifies the selected record, removes it from the list, saves the updated data, and refreshes the total.

4.7 Core Python Concepts Demonstrated

- Lists: Maintain multiple expense records in an ordered collection.
- Dictionaries: Store each expense as key-value pairs.
- File Handling: Read and write expense data for persistent storage.
- Tkinter: Build a graphical interface and handle user events.

5. FEATURES, TESTING AND RESULTS

5.1 Main Features

18. Add Expense: Enter a date, category, description, and amount.
19. View Expenses: Display previously entered expense records.
20. Total Calculation: Show the sum of recorded expense amounts.
21. File Storage: Preserve data in a local file between sessions.
22. Delete Expense: Remove an unwanted selected record.
23. Input Validation: Check required fields and ensure the amount is valid.
24. Graphical Interface: Use Tkinter widgets to make the application easy to operate.

5.2 Testing

The following test cases can be used to verify the main functions during implementation. Actual status should be marked after running the application.

Test Case	Expected Result	Status
Launch application	Main window opens	To test
Add valid expense	Record is added and total updates	To test
Enter invalid amount	Validation message is shown	To test
Save and reopen	Previously saved records load	To test
Delete selected expense	Record is removed and total updates	To test
Open without data file	Application starts with an empty list	To test

5. FEATURES, TESTING AND RESULTS (CONTINUED)

5.3 Expected Results

When implemented and tested, the application should allow users to add and review expense records, calculate total spending, and preserve records using local file storage. The Tkinter interface should provide a convenient way to interact with the program without entering commands in a terminal.

Successful testing should confirm that valid expense details are accepted, invalid amounts are handled safely, saved records can be loaded after restarting the program, and totals are recalculated after adding or deleting records.

5.4 Sample Expense Record

Date	Category	Description	Amount (₹)
26-10-2026	Food	Lunch	120
26-10-2026	Travel	Bus fare	40
26-10-2026	Education	Notebook	75

The sample records above illustrate the type of data the application can store. They are examples for documentation and do not represent actual user transactions.

5.5 Result Summary

The project brings together Python data structures, file operations, and GUI programming in one practical application. The final result depends on completing implementation and testing of the described functions.

6. CONCLUSION, FUTURE SCOPE AND REFERENCES

6.1 Conclusion

Personal Expense Tracker is a Python-based desktop application concept designed to help users manage everyday spending in an organized manner. It provides a central place to record expenses, review saved entries, and calculate total spending.

The project demonstrates four important Python concepts: Lists for maintaining collections of expenses, Dictionaries for representing each record, File Handling for saving and loading data, and Tkinter for creating the graphical user interface. These concepts show how basic programming tools can be combined to solve a practical problem.

The application can be useful for individuals who want a simple local expense-recording tool. With careful input validation and testing, it can provide a convenient starting point for more advanced personal finance software.

6.2 Future Scope

- Monthly and yearly expense reports.
- Budget limits and alerts when spending approaches a limit.
- Category-wise charts and visual summaries.
- Search, sort, and filter options.
- Export to CSV or PDF.
- Password protection and local data backup.
- Optional cloud synchronization across devices.

6.3 References

- Python Software Foundation. Python Documentation. https://docs.python.org/3/
- Python Software Foundation. tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html
- Python Software Foundation. Data Structures. https://docs.python.org/3/tutorial/datastructures.html
- Python Software Foundation. Reading and Writing Files. https://docs.python.org/3/tutorial/inputoutput.html
- Python Software Foundation. json — JSON encoder and decoder. https://docs.python.org/3/library/json.html